



Tecnicatura Universitaria en Inteligencia Artificial
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Universidad Nacional de Rosario

Procesamiento del Lenguaje Natural

Trabajo Práctico 1 - Parte 2

25/5/2025

Profesores:

- Juan Pablo Manson
- Alan Geary
- Constantino Ferrucci
- Sol Berger Dolores

Autora:

- Rizzotto, Camila

Introducción

Este trabajo se enmarca en la necesidad de implementar las diferentes herramientas vistas en la materia de Procesamiento del Lenguaje Natural hasta la unidad 3. Se realizan 6 problemas diferentes, en los que algunas de las tareas implican la clasificación y análisis de textos relacionados con Pradera, el juego de mesa que me tocó trabajar, enfocándome en la integración de técnicas como vectorización, análisis semántico, clasificación automática y minería de texto, para mejorar la búsqueda y organización del conocimiento.

Este informe está estructurado en varias secciones que reflejan cada uno de los ejercicios planteados. Se inicia con la explicación y desarrollo del análisis semántico de textos, seguido por la extracción y categorización de sustantivos para búsquedas filtradas, la segmentación multilingüe, el análisis de sentimientos aplicado a reseñas, y finalmente la generación y clasificación automática de consultas. En cada sección se describen las metodologías empleadas, los resultados obtenidos y las conclusiones relevantes para el avance del proyecto.

Todo el trabajo fue realizado en un notebook de [Colab](#).

Problema 1

En este apartado solamente se me solicitó la carga del repositorio asignado. En mi caso, como ya mencioné, me tocó el juego Pradera, y su repositorio se encuentra [aquí](#).

Metodología

Se utilizó la API de Google Drive junto con herramientas provistas por Google Colab para autenticar el acceso y automatizar la descarga de los documentos.

La metodología aplicada consistió en los siguientes pasos:

1. **Autenticación:** se utilizó `google.colab.auth` para autenticar al usuario y permitir el acceso al contenido de Google Drive.
2. **Inicialización del servicio:** se creó un cliente del servicio de Drive mediante `googleapiclient.discovery.build`, lo que permitió interactuar con la API para buscar y acceder a archivos.
3. **Descarga de archivos:** usando `MediaIoBaseDownload`, se descargaron los documentos de texto almacenados en el repositorio, almacenándolos localmente en el entorno de ejecución.

Problema 2

Aquí tenía que separar en fragmentos un texto extenso extraído y vectorizar cada uno con alguno de los modelos de embedding vistos en clases. Luego realizar un análisis de similitud de texto ingresando varias frases a buscar semánticamente y comparar distintas técnicas de distancias vistas en clases.

Metodología

Para abordar este problema, decidí utilizar el texto 'Mecanica_del_juego.txt', y se implementó:

1. **Procesamiento del texto:**
Se utilizó **spaCy** para dividir el texto en oraciones y manipular su estructura lingüística. Las frases resultantes sirvieron como base para la búsqueda semántica.
2. **Visualización semántica:**
Se empleó **displacy** para inspeccionar y visualizar la estructura sintáctica de algunas frases, ayudando a interpretar los resultados.
3. **Vectorización semántica:**
Cada oración y cada consulta fueron transformadas en vectores semánticos utilizando el modelo preentrenado **sentence-transformers/msmarco-MiniLM-L-6-v3**, que genera representaciones densas y contextuales.
4. **Comparación de métricas de similitud:**
Se definieron e implementaron tres métricas clásicas:
 - **Similitud de Jaccard**
 - **Coeficiente de Dice**
 - **Similitud de Levenshtein**

Para cada consulta, se compararon los resultados obtenidos con cada métrica sobre el conjunto de fragmentos, mostrando los *top-k* resultados por métrica y permitiendo evaluar diferencias.

5. **Visualización dimensional** (opcional):
Para una comprensión visual, se aplicó reducción de dimensionalidad con **t-SNE** sobre los embeddings vectorizados y se graficó el espacio semántico utilizando **Plotly**. Esto permitió observar agrupamientos y cercanías entre consultas y fragmentos.

Problema 3

Este ejercicio planteaba nuevamente trabajar con un texto extenso segmentado, pero incorporando una capa de análisis morfosintáctico y semántico. Específicamente, se solicitaba:

- Extraer los **sustantivos** de cada fragmento utilizando técnicas de POS tagging.
- Aplicar **reconocimiento de entidades** (NER) sobre esos sustantivos.
- Realizar una **búsqueda semántica**, pero filtrando y priorizando los fragmentos en función de la presencia de esos sustantivos.
- Comparar las métricas de similitud vistas en clase y justificar la más adecuada para este tipo de búsqueda especializada.

Por todo ello y por simplicidad, decidí seguir trabajando con lo que había ya generado en el problema 2: el mismo texto ya fragmentado.

Metodología

1. **Segmentación del texto:**
Como en el ejercicio anterior, se utilizó **spaCy** para dividir el texto en fragmentos u oraciones.
2. **Extracción de sustantivos (POS tagging):**
Se aplicó el análisis morfosintáctico con spaCy para identificar los **sustantivos** (NOUN y PROPN) en cada fragmento, permitiendo construir un subconjunto de contenido significativo enfocado en objetos, conceptos y entidades del dominio.
3. **Reconocimiento de entidades (NER):**
Con el mismo modelo de spaCy, se identificaron **entidades nombradas** (como organizaciones, fechas, locaciones, productos, etc.), lo cual aportó mayor semántica a la estructura de los fragmentos y posibilitó filtrar y comparar mejor.
4. **Filtrado semántico por sustantivos:**
Para cada consulta, se extrajeron sus sustantivos y se priorizaron aquellos fragmentos que compartían sustantivos relevantes o entidades con la consulta. Esto actuó como un primer filtro antes de calcular similitudes vectoriales.
5. **Vectorización y comparación de similitudes:**
Se reutilizó el modelo sentence-transformers/msmarco-MiniLM-L-6-v3 para convertir tanto los fragmentos como la consulta en vectores densos. Luego, se aplicaron las métricas de similitud textual (Jaro-Winkler, Dice y Levenshtein), y se analizaron sus resultados para determinar cuál era más precisa en este nuevo contexto.

Problema 4

Este ejercicio proponía abordar una tarea de **detección de idiomas** sobre una colección de archivos de texto. El objetivo era:

- Identificar automáticamente el idioma de cada texto.
- Almacenar la información relevante (nombre del archivo, idioma detectado, contenido, etc.) en un **DataFrame estructurado** para su posterior análisis o procesamiento.

Metodología

Carga de archivos:

Se utilizaron técnicas previas del ejercicio 1 para acceder al repositorio de archivos, asegurando que todos los textos estuvieran disponibles para ser procesados en bloque.

1. **Detección de idioma:**
Para identificar el idioma de cada archivo de forma automática, se utilizó una herramienta de **detección de idioma basada en modelos preentrenados**, como langdetect o langid. Estos detectores permiten analizar una cadena textual y predecir el idioma más probable.

2. **Construcción del DataFrame:**

A medida que se procesaban los archivos, se fue creando un **DataFrame de Pandas**, en el que cada fila contenía:

- Nombre del archivo.
- Idioma detectado.

3. **Verificación y limpieza:**

Se incluyeron verificaciones básicas para validar que el idioma detectado tuviera sentido (por ejemplo, corregir detecciones erróneas sobre textos difíciles que solo contenían URLs).

Problema 5

Este ejercicio consistía en realizar un análisis de sentimientos sobre un conjunto de reseñas escritas por usuarios, utilizando modelos preentrenados. Luego, sobre esa base, se debía construir un sistema de búsqueda semántica que permitiera encontrar reseñas similares a una consulta ingresada, filtrando además por el sentimiento detectado (positivo, negativo o neutro).

Metodología

1. **Preprocesamiento de reseñas:**

Se partió del conjunto de reseñas que obtuvimos del archivo 'comentarios.txt' normalizando su formato y aplicando minúsculas para facilitar el análisis textual.

2. **Análisis de sentimientos:**

Se utilizó un modelo preentrenado de análisis de sentimientos basado en BERT (como cardiffnlp/twitter-roberta-base-sentiment o modelos similares disponibles en HuggingFace).

Este modelo clasificó cada reseña en una de tres categorías:

- **Positiva**
- **Neutra**
- **Negativa**

El resultado se almacenó en una nueva columna del DataFrame junto con la reseña original, lo que permitió segmentarlas por sentimiento.

3. **Vectorización semántica:**

Las reseñas fueron **vectorizadas** usando un modelo de sentence-transformers como all-mpnet-base-v2 o msMarco-MiniLM-L-6-v3, generando embeddings densos que representan el significado de cada texto.

4. **Sistema de búsqueda semántica:**

Se creó una función que:

- Recibía una consulta textual y un filtro opcional por sentimiento.
- Vectorizaba la consulta usando el mismo modelo de embeddings.
- Calculaba la **similitud coseno** entre la consulta y todas las reseñas vectorizadas.

- Ordenaba los resultados por mayor similitud y devolvía las reseñas más cercanas semánticamente, respetando el filtro de sentimiento si estaba activo.

Problema 6

Se propuso generar un set de datos compuesto por más de 300 preguntas relacionadas a juegos de mesa o similares, y luego clasificarlas automáticamente según la fuente de datos más adecuada para responderlas.

Luego, se debía vectorizar estas consultas y entrenar un modelo de clasificación automática, aplicando distintos algoritmos, comparando resultados y justificando la elección del modelo final.

Metodología aplicada:

1. Generación y curación de preguntas:

Se construyeron más de **300 preguntas distintas**, tratando de cubrir un espectro variado y representativo de las tres categorías. Las preguntas fueron revisadas, normalizadas (por ejemplo, aplicando `.lower()`) y almacenadas en un DataFrame junto con su etiqueta manual.

2. Codificación de categorías:

Se asignó a cada categoría un valor numérico: {'estadísticas': 0, 'información': 1, 'relaciones': 2}. Esta codificación fue necesaria para entrenar los modelos de clasificación supervisada.

3. Vectorización de consultas:

Las preguntas fueron transformadas en vectores semánticos mediante el modelo sentence-transformers/all-mpnet-base-v2, obteniendo representaciones densas de cada consulta en un espacio semántico.

4. Entrenamiento de modelos de clasificación:

Se entrenaron y compararon distintos enfoques:

1. **Regresión logística multinomial:** como primer enfoque simple y robusto.
2. **Red neuronal densa:** con una arquitectura básica (una capa oculta) utilizando TensorFlow/Keras.

Se compararon las métricas de ambos modelos (precisión, clasificación por clase, matriz de confusión).

5. Evaluación de desempeño:

Se utilizaron métricas como **accuracy** y **matriz de confusión** para evaluar los modelos.

Se concluyó que ambos modelos funcionaban razonablemente bien, aunque la red neuronal mostró una leve mejora en la precisión en categorías más complejas como "relaciones".

6. Pruebas con nuevas frases:

Se desarrolló una función que permite **ingresar nuevas preguntas** y obtener su categoría predicha automáticamente, como parte de una aplicación práctica del modelo entrenado.

Conclusiones

A lo largo de este trabajo se abordaron distintos problemas de procesamiento de lenguaje natural aplicados al dominio de juegos y reseñas, explorando técnicas tanto clásicas como modernas. Se utilizaron modelos de embeddings semánticos, análisis de similitud textual, clasificación supervisada, análisis de sentimientos y detección de idioma, aplicando herramientas como spaCy, sentence-transformers, scikit-learn y redes neuronales.

Se construyeron datasets propios, se evaluaron diferentes métricas y se compararon enfoques para resolver tareas concretas como la categorización de preguntas o la búsqueda semántica filtrada. El abordaje integral y comparativo permitió no solo cumplir con los objetivos propuestos, sino también extraer aprendizajes relevantes sobre el desempeño de distintas metodologías en problemas reales de PLN.